
Artificial Intelligence Fundamentals

Search, evolutionary optimization and artificial life, logical and probabilistic agents

Francesco Secoli • Academic year 2026/2027

Disclaimer. Derived from the *Artificial Intelligence Fundamentals* course materials (Academic Year 2026/2027), MSc in Data Science & Business Informatics, University of Pisa.

These notes are open educational content created by a student. They are **not an academic source** and may contain inaccuracies. You may freely share, modify, and reuse this material for educational and non-commercial purposes with appropriate attribution. The content is my personal interpretation of the professor's course materials and should not replace official teaching resources. I assume no responsibility for any errors or misinterpretations.

These notes were produced with an **AI-in-the-middle** workflow: a first human pass, then **Claude Code** to support formulation, understanding, and rewriting, followed by a final human review.

If you find errors, have suggestions, or spot unintentionally included copyrighted material (which I will promptly remove on notification), contact me at sclfnc@proton.me.

Contents

1	Introduction	1
1.1	Symbolic AI	1
1.2	The symbol grounding problem	1
1.3	The physical symbol system hypothesis	2
1.4	Intelligence is not consciousness	2
1.5	Sub-symbolic representations	3
1.6	Models of the world	3
1.7	Hybrid systems	3
2	Intelligent agents	4
2.1	Agents and environments	4
2.2	Rationality	5
2.3	Fixed action patterns	6
2.4	Task environments: PEAS	6
2.5	Properties of environments	7
2.6	Types of agent	7
2.7	Knowledge representation	9

1 Introduction

Artificial intelligence (AI) is a large family of methods. Machine learning (ML) is one member of the family, deep learning (DL) is a branch of machine learning, and the large language models (LLMs) behind tools such as ChatGPT and Copilot are a branch of deep learning. This course covers, in the lecturer's words, "all AI, but learning": methods that answer questions broader than those of machine learning and that run, mostly unseen, inside many AI-based tools. Some learning may still appear along the way, and Section 1.7 gives the reason for this choice.

After a lecture on the design of intelligent agents (Section 2), the course runs in four blocks:

- **Solving problems by search:** uninformed and informed search, local search, constraint satisfaction problems, search in games.
- **Evolutionary optimization and artificial life:** evolutionary optimization, novelty-based search and quality-diversity, artificial life.
- **Logical agents:** inference in propositional and first-order logic, planning.
- **Probabilistic agents:** Bayesian networks, hidden Markov models.

Multi-agent systems close the course.

1.1 Symbolic AI

Symbolic AI was the dominant paradigm of the field from the 1950s to the 1990s. It represents the world at a high level, in a form that humans can read and interpret, and its building block is the symbol.

Definition 1.1 (Symbol). A **symbol** is a meaningful pattern that can be manipulated.

A symbol stands for something else, as the string `cat` stands for the animal, and a program can operate on it: compare it, copy it, combine it with other symbols into rules. Semiotics, the study of signs and of how they carry meaning, names the two sides of this relation: the pattern is the **signifier**, and what it stands for is the **signified**.

Where do the symbols of a machine come from? The lecture gives two answers. Either humans provide them, and then the machine's symbols mean something only because someone who already had symbols gave them their meaning: a chicken-and-egg problem. Or the computer builds its own low-level representations from raw data, which is sub-symbolic manipulation (Section 1.5). The lecturer relates this circularity to Hofstadter's "strange loops".

1.2 The symbol grounding problem

If a symbol means something only to the humans who supplied it, the system that manipulates the symbol does not know what it manipulates. Harnad named this difficulty.

Definition 1.2 (Symbol grounding problem). The **symbol grounding problem** asks how a symbol is connected to the real-world object it stands for: what makes the connection, who makes it, and how. In Harnad's words: "How can the semantic interpretation of a formal

symbol system be made intrinsic to the system, rather than just parasitic on the meanings in our heads?”¹

A symbol represents something outside the system, so the answer requires what Hofstadter calls “jumping out of the system”. Formal logic meets the same limit in Gödel’s incompleteness theorem: every consistent, effectively axiomatized formal system that can express elementary arithmetic contains a true statement that it cannot prove, so establishing that statement requires a step outside the system.

1.3 The physical symbol system hypothesis

Suppose that symbols can be grounded. Is manipulating them then enough for intelligence? Newell and Simon answered yes, in the form of a hypothesis about a particular kind of system.

Definition 1.3 (Physical symbol system). A **physical symbol system** is a system that manipulates symbols to produce other symbols. It is *physical* because the manipulation is a physical process, which consumes energy.

In the lecture’s terms, humans are physical symbol systems, and so are digital computers, since both manipulate symbols. The **physical symbol system hypothesis** (PSSH) states:

A physical symbol system has the necessary and sufficient means for general intelligent action.²

Newell and Simon explain both halves. *Necessary* means that “any system that exhibits general intelligence will prove upon analysis to be a physical symbol system”. *Sufficient* means that “any physical symbol system of sufficient size can be organized further to exhibit general intelligence”. The claim is empirical, which is why it is a hypothesis and not a theorem: evidence can support it or refute it. If it holds, a digital computer, suitably organized, can act with general intelligence.

1.4 Intelligence is not consciousness

A system that acts intelligently need not be conscious: intelligence and consciousness are different properties. Consciousness is usually tied to subjective experience. In the sense of Nagel’s essay “What is it like to be a bat?”,³ a being is conscious if there is something it is like to be that being. Subjective experience is hard to detect from outside: dissecting a system and studying its behavior may not be enough. A **zombie**, which behaves exactly like a person but has no experience, would pass every behavioral test. Searle’s Chinese room makes a related point about understanding: a person who follows written rules to manipulate Chinese characters can answer questions in Chinese without understanding any Chinese.⁴

If consciousness exists, materialism holds that it is a by-product of the body, located “somewhere” in it; dualism instead treats the mind as a substance different from the body. Hofstadter argues

¹S. Harnad, “The symbol grounding problem”, *Physica D* 42 (1990), pp. 335–346.

²A. Newell and H. A. Simon, “Computer science as empirical inquiry: symbols and search”, *Communications of the ACM* 19(3) (1976), pp. 113–126.

³T. Nagel, “What is it like to be a bat?”, *The Philosophical Review* 83(4) (1974).

⁴J. R. Searle, “Minds, brains, and programs”, *Behavioral and Brain Sciences* 3(3) (1980).

that what matters is the symbolic pattern and not its physical substrate: silicon chips organized in an equivalent way could support consciousness just as neurons do. The lecture leaves the question open.

1.5 Sub-symbolic representations

Symbolic descriptions break down when the world varies more than a set of rules can anticipate. The alternative is to let the machine build its own representation.

Definition 1.4 (Sub-symbolic representation). A **sub-symbolic representation** encodes a concept as a numeric pattern spread over many low-level units, such as the weights of a neural network, instead of as a single symbol that a human can read.

Example 1.5 (Recognizing a cat). This example contrasts the two approaches. A symbolic system describes a cat through symbols for its parts, such as two eyes, whiskers and pointed ears, and through rules that combine them. Real photographs of cats vary in pose, breed, lighting and framing, and each new variation breaks some rule. A sub-symbolic system learns its representation from many example images and absorbs the variation, at the price of a representation that no human can read.

1.6 Models of the world

Whatever its representation, an AI system reasons about a **model of the world**: a representation of the part of reality that the system works on. The lecture distinguishes three kinds of model, by the question that each answers.

- **Descriptive**: how is the world? A map of Pisa describes where things are.
- **Predictive**: what value will an unobserved quantity take? A model estimates the selling price of a house from its floor area, its number of rooms and its energy efficiency. Machine learning builds models of this kind.
- **Decisional**: what should be done to reach a goal? A path through a labyrinth answers it; the search methods of the course answer questions of this kind.

1.7 Hybrid systems

Machine learning maps inputs X to outputs Y . Building the map (training) requires huge amounts of data, and the more complex the mapping, the more data it requires. Using the map (inference) requires expensive memory, time and dedicated accelerators such as GPUs and TPUs. Deep learning requires even more data and computation. It still works very well in many applications, and the lecture states the course's position explicitly: "This course is not saying 'don't use deep learning'. It's saying 'don't use deep learning only'." A **hybrid system** combines a learned model with components that are not learned.

Example 1.6 (Retrieval-augmented generation). Retrieval-augmented generation (RAG) is a worked case of a hybrid system. A **retriever** fetches the documents relevant to a question from an external memory, such as Wikipedia, and a **generator**, a language model, writes the

answer from the question and those documents. We can expand the memory to a new domain, or update it with the news, without retraining the model, and each answer can point to the document it comes from, which makes it easier to interpret.

2 Intelligent agents

The course builds AI systems as agents: systems that perceive an environment and act on it. This section fixes the vocabulary for the rest of the course. We define what an agent is, when its behavior counts as rational, how to describe the task it faces and its environment, which designs of agent exist, and how an agent can represent what it knows.

2.1 Agents and environments

To reason about any AI system we first separate the part that decides from the world it decides about.

Definition 2.1 (Agent). An **agent** is anything that can act. It perceives its **environment** through **sensors** and acts on it through **actuators**.

Where the agent ends and the environment begins is a choice of the designer, and the agent must be delimited somehow before we can study it. What the sensors deliver at one instant is a **percept**. Time runs in discrete steps $t = 1, 2, \dots$, and at each step the agent receives one percept, so its experience accumulates as a sequence of percepts.

Definition 2.2 (Agent function). The **agent function**, or **policy**, is a map

$$f: P^* \rightarrow A,$$

where P^* is the set of percept histories, the finite sequences of percepts, and A is the set of the agent's actions.

In words: the agent function says which action the agent takes after every possible history of percepts. We can implement f physically, in a robot, or simulate it, in a program.

Example 2.3 (Vacuum-cleaner world). This worked example runs through the rest of the section. The world has two squares, A and B, and the agent is a vacuum cleaner.

- The actions are $A = \{Left, Right, Suck, NoOp\}$.
- The position of the agent is $x \in \{A, B\}$.
- The status of the square that the agent occupies is $s_t \in S = \{clean, dirty\}$. The agent knows it only through perception.
- The percept at time t is the pair (x_t, s_t) .
- The task is to clean the environment.

Building an agent means building f . Algorithm 1 is a first attempt: it chooses from the current percept alone. It learns nothing and contains no AI, because the designer wrote down every case and its action by hand.

Algorithm 1 Reflex vacuum agent

Require: the percept (*location, status*)**Ensure:** an action

```

1: function REFLEX-VACUUM-AGENT(location, status)
2:   if status = dirty then
3:     return Suck
4:   else if location = A then
5:     return Right
6:   else if location = B then
7:     return Left
8:   end if
9: end function

```

2.2 Rationality

Intuitively, Algorithm 1 makes sense. To say that it is a *good* agent, though, we need a criterion. Most definitions of AI ask the agent to do “the right thing”. They do not mention intelligence at all: this narrows the scope of the field, but it makes the goal precise enough to build agents toward it.

Moral philosophy offers several notions of the right thing. AI adopts **consequentialism**: we evaluate an agent by the consequences of its actions. What counts is the sequence of environment states that its behavior produces, not the agent’s intentions or its own opinion of its success. The choice is natural for a machine, because consequences can be observed and measured while intentions cannot. It also comes with a lot of work, since we must now say which consequences are good. In a simple world we could list them. As the environment grows more complex, no complete evaluation of every behavior and its results is possible, so we summarize the good consequences in a single number.

Definition 2.4 (Performance measure). A **performance measure** evaluates the goodness of an environment state, or of a sequence of environment states.

Definition 2.5 (Rational agent). For each possible percept sequence, a **rational agent** selects an action that is expected to maximize its performance measure, given the evidence provided by the percept sequence and whatever built-in knowledge the agent has.

In words: a rational agent does the best it can with what it has perceived and what it knew from the start, where “best” is fixed by the performance measure. The summary is where the difficulty lies, as Goodhart’s law warns: “when a measure becomes a target, it ceases to be a good measure”.

Example 2.6 (A measure that backfires). This example shows a measure failing. Suppose that we cannot enter the room to check whether it is clean, so we reward the vacuum cleaner with the grams of dirt that it picks up over a one-hour shift. An agent that maximizes this measure can dump the dirt back on the floor and pick it up again: its score keeps growing and the room is never cleaner. The measure rewarded a behavior that we expected to be useful,

instead of the state of the environment that we actually wanted.

Rationality also asks less than perfection. A rational agent maximizes *expected* performance given the percept history, so:

- **rational is not omniscient**: the percepts may not supply all the relevant information;
- **rational is not clairvoyant**: the agent has no access to future percepts;
- **rational is not successful**: the outcome of an action may not be the expected one.

In every case, careful engineering of the performance measure, and control of the behavior that the agent learns, are key.

2.3 Fixed action patterns

An environment that is completely observable and predictable never changes its optimal solution, and so it encourages the agent to learn one fixed behavior. The result is a fragile agent that follows a **fixed action pattern**: it keeps executing its policy even when the world changes under it.

Example 2.7 (The Sphex wasp). Nature provides a contrast case. After digging a burrow, the Sphex wasp goes out, stings a caterpillar and drags it to the burrow. It enters the burrow to check that all is well, drags the caterpillar inside and lays its eggs. If an entomologist moves the caterpillar a few inches away while the wasp is checking the burrow, the wasp goes back to the “drag the caterpillar” step and continues its plan without modification, checking the burrow again. It does so even after dozens of such interventions.

2.4 Task environments: PEAS

Before designing an agent we describe the task that it must solve. A **PEAS** description lists four elements: the **P**erformance measure, the **E**nvironment, the **A**ctuators and the **S**ensors. The actuators and the sensors form the **agent architecture**, and the agent is the architecture together with the agent function.

Example 2.8 (Autonomous taxi). Table 2.1 is a worked PEAS description.

Table 2.1. PEAS description of an autonomous taxi service.

Element	Autonomous taxi
Performance measure	safety, reaching the destination, profits, legality, comfort, ...
Environment	streets and freeways, traffic, pedestrians, weather, ...
Actuators	steering, accelerator, brake, horn, speaker or display, ...
Sensors	cameras, accelerometers, gauges, engine sensors, keyboard, GPS, ...

The more restricted the environment, the easier the design of the agent.

2.5 Properties of environments

Which agent to build depends on the environment, so we define the environment first. We classify it along four dimensions, plus a fifth that concerns what the agent knows.

- **Observable.** The environment is **fully observable** if the sensors give access to its complete state, **partially observable** if faulty sensors or missing information hide part of it, and **unobservable** if the agent has no sensor information at all.
- **Deterministic.** The environment is **deterministic** if the current state and the action uniquely determine the next state, and **nondeterministic** (or stochastic) if the same state and action can lead to different next states. Partial observability is one possible cause of nondeterminism.
- **Discrete.** The state of the environment, time, the percepts and the actions can each take **discrete** or **continuous** values.
- **Single agent.** The environment is **multi-agent** if the performance of our agent depends on the actions of other agents, and **single-agent** otherwise.
- **Known.** In a **known** environment the agent knows its rules, that is, which next state follows from each state and action. A known environment can still be partially observable. In an **unknown** environment the agent may try to learn the rules.

Table 2.2 classifies three environments. The real world sits in the hardest column: like the taxi, it is partially observable, nondeterministic, continuous and multi-agent.

Table 2.2. Three environments along the four dimensions.

Characteristic	Sudoku	Backgammon	Taxi service
Fully observable	yes	yes	no
Deterministic	yes	no	no
Discrete	yes	yes	no
Single-agent	yes	no	no

2.6 Types of agent

Agents differ in how they compute the agent function. We go from the simplest design to the most flexible one; each design adds a component that removes a limit of the previous one.

2.6.1 Table-driven agents

The most direct way to build f is to write it down. A **table-driven agent** stores its whole percept history and looks up the action in a table indexed by percept sequences (Algorithm 2).

The design is only feasible in simple environments. With a set P of possible percepts, the table needs one entry for each percept sequence, that is, $\sum_{t=1}^T |P|^t$ entries for an agent that lives T steps. The complexity of the environment and of the agent makes the table far too large: the design trades efficiency for effectiveness.

Algorithm 2 Table-driven agent

Require: a percept**Ensure:** an action**persistent:** *percepts*, a sequence, initially empty; *table*, a table of actions indexed by percept sequences, initially fully specified

```

1: function TABLE-DRIVEN-AGENT(percept)
2:   append percept to the end of percepts
3:   action  $\leftarrow$  LOOKUP(percepts, table)
4:   return action
5: end function

```

2.6.2 Simple reflex agents

A **simple reflex agent** chooses its action from the current percept alone, through **condition–action rules**: if the percept satisfies a condition, then the agent takes the corresponding action. The name comes from the body: like a reflex, the agent reacts to the stimulus without consulting any memory. Algorithm 1 is a simple reflex agent.

The simplification cuts the cost dramatically. The table of a table-driven agent grows exponentially with the lifetime T , while a simple reflex agent needs one rule per percept, and its work over T steps grows linearly with T . It simplifies an already naive approach, though: it only works in simple environments, and small, fully observable environments are rare.

2.6.3 Model-based agents

A simple reflex agent fails as soon as the current percept does not reveal the state. A **model-based agent** keeps an **internal state**, which depends on the percept history, and uses it to estimate the current state of the environment. Two models update the internal state:

- the **transition model** describes how the world evolves, and in particular the consequences of the agent’s actions;
- the **sensor model** describes how the state of the world shows up in the agent’s percepts.

By approximating the entire state, a model-based agent handles partially observable environments well. Acquiring such a model is still hard: the lecture notes that large language models struggle to acquire a model of the world.⁵

2.6.4 Goal-based agents

Knowing the state is not enough to know what to do. A **goal-based agent** also represents **goals**, descriptions of the states it wants to reach, and it predicts what the world will be like if it takes an action. To choose, it must weigh the immediate reward of an action against the expected long-term reward. Two families of methods explore the options: **search**, which navigates the tree of possible decisions, and **planning**, which exploits knowledge about the world to execute the best actions. Later blocks of the course study both.

The comparison with a reflex agent shows what goals buy. A reflex agent has no notion of a goal:

⁵The slide points to M. Mitchell, “LLMs and world models, part 1”.

its condition–action rules model a single problem, so a new goal means rebuilding the agent from scratch. A goal-based agent represents its goals explicitly and can change them.

2.6.5 Utility-based agents

A goal only separates success from failure. A **utility-based agent** assigns a value, its **utility**, to every intermediate state or sequence of states, not only to the final goal: utility works as an internal performance measure. Utility-based agents are often used in stochastic, partially observable environments. There the agent maximizes, for each action, the **expected utility** over the possible results i of the action:

$$EU = \sum_i p_i u_i,$$

where p_i is the probability of result i and u_i is its utility.

2.6.6 Learning agents

Every design so far is fixed by its designer. A **learning agent** improves itself through four components:

- the **policy** chooses an action, given the context (the percepts and the internal state);
- the **critic** evaluates the current behavior against an external performance measure and sends feedback;
- the **learning element** uses the feedback to change the other components;
- the **problem generator** steers the agent away from greedy behavior by trying out new actions, which balances the critic.

Learning is orthogonal to the previous types: any of them can be a learning agent or not, and learning can modify any of its components.

2.7 Knowledge representation

Every agent above represents the world somehow. The lecture places a representation on two axes.

Expressiveness. An **atomic** representation treats each state as indivisible, with no internal structure. A **factored** representation describes a state as a vector of attribute values. A **structured** representation describes a state as a graph: objects with their properties and the relationships among them. Expressiveness grows from atomic to structured.

Distribution. In a **localist** representation, concepts and memory locations correspond one to one. In a **distributed** representation, one memory cell takes part in several concepts, and one concept is spread over several cells.